

MODUL 9

API PERANGKAT KERAS

1. Mengimplementasikan Akses Kamera dan Media API (Image Picker) pada Aplikasi Flutter, Serta Menangani Konfigurasi Perizinan Privasi Perangkat (Info.plist pada iOS dan Minimum SDK pada Android).

- a. Code

➤ Main.dart

```
import 'dart:async';
import 'dart:io';

import 'package:camera/camera.dart';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

List<CameraDescription> cameras = [];

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  try {
    cameras = await availableCameras();
  } on CameraException catch (e) {
    debugPrint('Error: ${e.code}\nError Message: ${e.description}');
  }
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hardware API App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Hardware API (Modul 9)'),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          ElevatedButton.icon(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const
CameraApp()),
              );
            },
            icon: const Icon(Icons.camera),
            label: const Text('Camera API (Live Preview)'),
          ),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => const
ImageFromGalleryEx(ImageSourceType.camera),
                ),
            );
          },
          icon: const Icon(Icons.add_a_photo),
          label: const Text('Image Picker (Camera)'),
        ),
        const SizedBox(height: 20),
        ElevatedButton.icon(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const
ImageFromGalleryEx(ImageSourceType.gallery),
              ),
            );
          },
          icon: const Icon(Icons.photo_library),
          label: const Text('Image Picker (Gallery)'),
        ),
      ],
    ),
  );
}

```

```

        ],
    ),
),
);
}
}

// =====
// BAGIAN 1: IMPLEMENTASI CAMERA API
// =====
class CameraApp extends StatefulWidget {
  const CameraApp({Key? key}) : super(key: key);

  @override
  _CameraAppState createState() => _CameraAppState();
}

class _CameraAppState extends State<CameraApp> {
  CameraController? controller;
  XFile? imageFile;

  @override
  void initState() {
    super.initState();
    if (cameras.isNotEmpty) {
      controller = CameraController(cameras[0], ResolutionPreset.max);
      controller!.initialize().then((_) {
        if (!mounted) {
          return;
        }
        setState(() {});
      }).catchError((Object e) {
        if (e is CameraException) {
          switch (e.code) {
            case 'CameraAccessDenied':
              debugPrint('User denied camera access.');
```

```

        super.dispose();
    }

    void onTakePictureButtonPressed() async {
        if (controller == null || !controller!.value.isInitialized) {
            return;
        }
        if (controller!.value.isTakingPicture) {
            return;
        }

        try {
            XFile file = await controller!.takePicture();
            setState(() {
                imageFile = file;
            });
        } on CameraException catch (e) {
            debugPrint('Error occurred while taking picture: $e');
        }
    }

    @override
    Widget build(BuildContext context) {
        if (controller == null || !controller!.value.isInitialized) {
            return const Scaffold(
                body: Center(child: Text('Initializing Camera...')),
            );
        }
        return Scaffold(
            appBar: AppBar(
                title: const Text('Camera API Preview'),
            ),
            body: Column(
                children: [
                    Expanded(
                        child: Padding(
                            padding: const EdgeInsets.all(8.0),
                            child: Center(
                                child: CameraPreview(controller!),
                            ),
                        ),
                    ),
                    if (imageFile != null)
                        Expanded(
                            child: Padding(
                                padding: const EdgeInsets.all(8.0),
                                child: Image.file(
                                    File(imageFile!.path),
                                    fit: BoxFit.cover,
                                ),
                            ),
                        ),
                ],
            ),
        );
    }

```

```

        ),
    ),
),
],
),
floatingActionButton: FloatingActionButton(
    onPressed: onTakePictureButtonPressed,
    child: const Icon(Icons.camera_alt),
),
);
}
}

// =====
// BAGIAN 2: IMPLEMENTASI MEDIA API (IMAGE PICKER)
// =====
enum ImageSourceType { camera, gallery }

class ImageFromGalleryEx extends StatefulWidget {
    final ImageSourceType type;

    const ImageFromGalleryEx(this.type, {Key? key}) : super(key: key);

    @override
    ImageFromGalleryExState createState() =>
ImageFromGalleryExState(type);
}

class ImageFromGalleryExState extends State<ImageFromGalleryEx> {
    File? _image;
    late ImagePicker imagePicker;
    final ImageSourceType type;

    ImageFromGalleryExState(this.type);

    @override
    void initState() {
        super.initState();
        imagePicker = ImagePicker();
    }

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text(type == ImageSourceType.camera
                    ? "Image from Camera"
                    : "Image from Gallery")),
            body: Column(
                children: <Widget>[

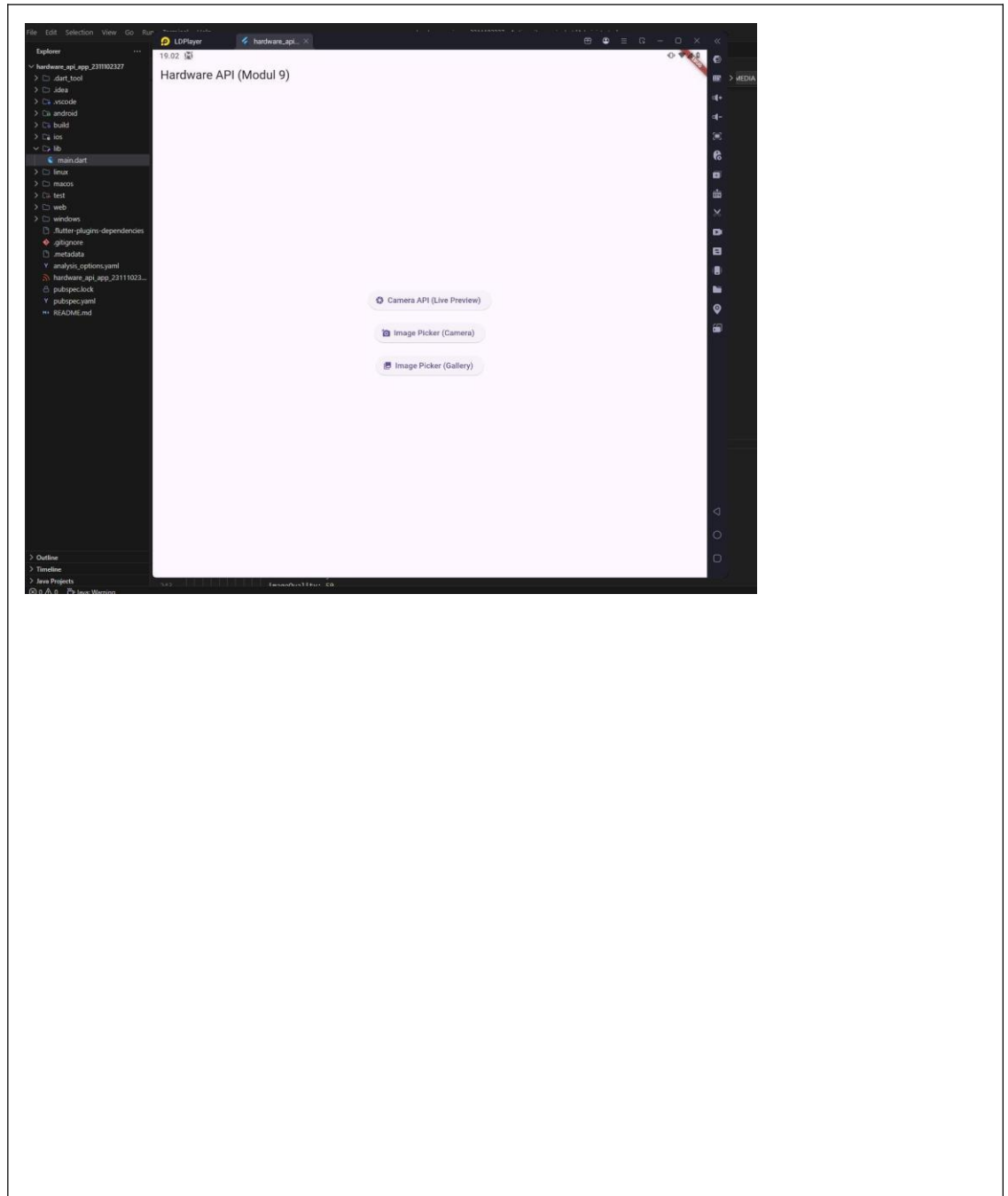
```

```

const SizedBox(
  height: 52,
),
Center(
  child: GestureDetector(
    onTap: () async {
      var source = type == ImageSourceType.camera
        ? ImageSource.camera
        : ImageSource.gallery;
      XFile? image = await imagePicker.pickImage(
        source: source,
        imageQuality: 50,
        preferredCameraDevice: CameraDevice.front);
      if (image != null) {
        setState(() {
          _image = File(image.path);
        });
      }
    },
    child: Container(
      width: 200,
      height: 200,
      decoration: BoxDecoration(color: Colors.red[200]),
      child: _image != null
        ? Image.file(
            _image!,
            width: 200.0,
            height: 200.0,
            fit: BoxFit.fitHeight,
          )
        : Container(
            decoration: BoxDecoration(color:
Colors.red[200]),
            width: 200,
            height: 200,
            child: Icon(
              Icons.camera_alt,
              color: Colors.grey[800],
            ),
          ),
    ),
  ),
),
),
],
),
);
}
}

```

b. *Screenshot output*



c. Penjelasan dari code dan output (korelasi)

Aplikasi Flutter ini dirancang dengan struktur yang sistematis untuk mengintegrasikan fitur perangkat keras, khususnya kamera dan galeri foto, ke dalam sistem aplikasi mobile. Pada file **main.dart**, proses dimulai melalui fungsi asinkron **main()** yang berperan sebagai titik awal aplikasi. Fungsi ini menjalankan **WidgetsFlutterBinding.ensureInitialized()** guna memastikan framework Flutter telah siap digunakan, kemudian melakukan inisialisasi pustaka kamera melalui **availableCameras()**. Seluruh kamera yang terdeteksi pada perangkat akan disimpan dalam variabel global **cameras**, sehingga dapat diakses dengan cepat oleh berbagai halaman dalam aplikasi.

Selanjutnya, pengelolaan tampilan utama dan perpindahan halaman dilakukan melalui kelas **HomeScreen** dan **CameraScreen**. Pada halaman **HomeScreen**, antarmuka dibangun menggunakan widget **Scaffold** yang menampilkan tiga tombol utama di tengah layar, yaitu “**Go to Camera**”, “**Select Image from Gallery**”, dan “**Take Image from Camera**”. Tombol pertama digunakan untuk membuka fitur kamera secara langsung, sedangkan dua tombol lainnya memanfaatkan objek **ImagePicker()** untuk memilih gambar dari galeri atau mengambil foto menggunakan kamera. Saat salah satu tombol tersebut ditekan, fungsi **pickImage** akan dijalankan dengan kualitas gambar yang dikompresi hingga 50% guna menghemat penggunaan memori. File gambar yang dipilih kemudian disimpan secara asinkronus ke dalam variabel state **_image**.

Hubungan antara kode program dan tampilan aplikasi terjadi secara dinamis melalui mekanisme **state management**. Ketika pengguna memilih tombol kamera langsung, fungsi **Navigator.push** akan mengarahkan aplikasi menuju halaman **CameraScreen**. Pada halaman tersebut, objek **CameraController** diinisialisasi menggunakan kamera depan (**CameraDevice.front**) atau kamera utama dengan pengaturan resolusi tertinggi (**ResolutionPreset.max**). Setelah proses inisialisasi selesai, widget **CameraPreview** akan menampilkan hasil tangkapan kamera secara real-time pada layar. Sebaliknya, apabila pengguna menggunakan fitur **Image Picker**, widget **Image.file(_image!)** pada halaman utama akan secara otomatis memperbarui tampilan sehingga area yang sebelumnya kosong berubah menjadi pratinjau gambar yang telah dipilih dari galeri atau hasil foto yang baru diambil.